

Informe de Resultados

Análisis de Resultados de Prueba de Carga (Ejercicio 2)

Kris Olalla

1. Resumen ejecutivo

Se analizó el resultado de una ejecución de prueba de carga (archivo textSummary.txt de k6) junto con su gráfico de monitoreo de usuarios virtuales (VUs) frente a peticiones por segundo (http_reqs). El objetivo es interpretar las métricas obtenidas, contrastarlas contra los criterios de aceptación del reto (SLA: ≥ 20 TPS, tiempo de respuesta $\leq 1.5s$, tasa de error $< 3\%$) y emitir hallazgos, conclusiones y recomendaciones basadas en evidencia.

Criterio SLA	Umbral	Resultado observado	Veredicto
Throughput (TPS)	≥ 20 TPS	73.18 req/s (promedio total)	CUMPLE (con salvedad)
Tasa de error	$< 3\%$	2.44 % (agregado)	CUMPLE EN AGREGADO
Tiempo de respuesta	$p(95) \leq 1.5$ s	$p(90)=1.28s \cdot p(95)=1.57s$	NO CUMPLE

Veredicto general: la ejecución analizada cumple parcialmente el SLA definido. El throughput y la tasa de error global están dentro de los umbrales exigidos, pero el percentil 95 del tiempo de respuesta (1.57s) supera el máximo permitido (1.5s), y el análisis por etapas revela una concentración casi total (99.96%) de los errores durante el tramo de carga sostenida (estado estable), coincidiendo con una caída visible de VUs y throughput en el gráfico de monitoreo. Esto indica un incidente de disponibilidad puntual que el promedio agregado oculta.

2. Alcance y contexto de los datos analizados

Los datos de este análisis corresponden a los insumos entregados en el enunciado del reto (Figura 1: textSummary.txt, y Figura 2: gráfico de monitoreo VUs vs. http_reqs), no a una ejecución realizada por el equipo autor de este informe. El check reportado en el resumen ("App Transaction Balance response was OK") corresponde a una transacción distinta al servicio de login trabajado en el Ejercicio 1; se usa aquí únicamente como caso de análisis de un resultado real de k6, tal como lo solicita el enunciado del Ejercicio 2. Los principios de interpretación aplicados (percentiles, correlación VUs/throughput, análisis por etapas) son igualmente válidos y se recomienda aplicarlos sobre cualquier ejecución futura del servicio de login.

3. Métricas generales de la ejecución

Métrica	Valor
Peticiones totales (http_reqs)	276,650
Duración aprox. de la prueba	~63 min (276,650 / 73.18 req/s)
Throughput promedio	73.176857 req/s
Iteraciones	276,650 (73.176857/s)
Checks exitosos	269,891 (97.55%)
Checks fallidos	6,759 (2.44%)
VUs (min / max)	2 / 140
Datos recibidos	842 MB (223 kB/s)
Datos enviados	588 MB (156 kB/s)

3.1 Nota sobre el throughput promedio

El promedio de 73.18 req/s corresponde a toda la ventana de ejecución (~63 minutos) e incluye, muy probablemente, tramos de rampa de subida y rampa de bajada de menor intensidad (igual que se documentó en el Ejercicio 1). Al estar muy por encima del umbral de 20 TPS, el margen es suficientemente amplio como para asumir con confianza que el estado estable también lo supera; sin embargo, no se cuenta aquí con la serie temporal detallada (segundo a segundo) para confirmarlo con la misma precisión que en el Ejercicio 1 — se recomienda exportar y conservar esa serie en futuras pruebas.

4. Análisis de tiempos de respuesta y percentiles

Métrica	avg	min	med	max	p(90)	p(95)
http_req_duration (todas)	861.68ms	191.86ms	613.42ms	29.93s	1.28s	1.57s
http_req_duration (exp:true)	735.84ms	244.92ms	600.7ms	26.72s	1.22s	1.42s
iteration_duration	1.86s	0s	1.61s	30.94s	2.29s	2.57s

El percentil 95 (1.57s) excede el umbral de 1.5s definido en el SLA, mientras que el p(90) (1.28s) sí lo cumple. Esto significa que aproximadamente entre el 5% y el 10% de las peticiones (del orden de 14,000 a 27,000 sobre 276,650) tardaron más de 1.28–1.57 segundos, y una fracción de ellas llegó a tardar hasta 29.93 segundos — un valor extremo, ~20 veces el límite del SLA, típico de timeouts o cuelgues en el backend o en una dependencia downstream, no de latencia de red normal.

Es notable que el promedio de las peticiones exitosas (expected_response:true, avg=735.84ms) es considerablemente menor que el promedio general (861.68ms). Esto confirma que las peticiones fallidas no solo aportan error de negocio, sino que además son, en promedio, mucho más lentas (muy probablemente por timeouts esperando una respuesta que nunca llega dentro de los límites normales), arrastrando el promedio y los percentiles altos hacia arriba.

5. Análisis de errores por etapa

Etapa	HTTP 4xx	HTTP 5xx	Total fallos	% del total
Stage 0 (rampa de subida)	0	1	1	0.01%
Stage 1 (estado estable, ~140 VUs)	769	5,987	6,756	99.96%
Stage 2 (rampa de bajada)	0	2	2	0.03%
Total	769	5,990	6,759	100%

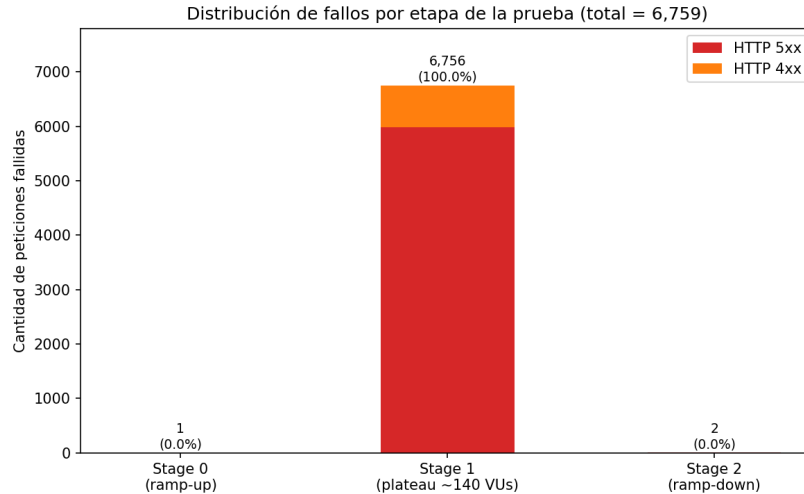


Figura A. Distribución de fallos por etapa de la prueba (elaboración propia a partir de las métricas custom del textSummary.txt).

El 99.96% de los 6,759 fallos ocurre durante la etapa de carga sostenida (estado estable, ~140 VUs concurrentes), no durante la rampa de subida ni la rampa de bajada. Esto es un patrón característico de problemas de capacidad que solo se manifiestan bajo concurrencia sostenida (agotamiento de pools de conexiones, saturación de hilos/workers, degradación de una dependencia downstream, backpressure acumulado, o efectos de garbage collection), y descarta que el problema esté relacionado con la velocidad de incremento/decremento de carga en sí.

Adicionalmente, la tasa de error de 2.44% reportada es un promedio sobre las 276,650 peticiones de toda la prueba. Si el incidente se concentró en una ventana de pocos minutos dentro del estado estable (ver sección 6), la tasa de error real durante esa ventana pudo haber sido drásticamente más alta que el 2.44% agregado — posiblemente cercana o superior al 3% del SLA, o incluso mucho mayor de forma localizada — quedando enmascarada por el resto de la prueba, que sí operó de forma estable. Este es el mismo tipo de distorsión por promediado identificado en el Ejercicio 1 al comparar el TPS de estado estable contra el TPS promedio de toda la corrida.

6. Correlación entre usuarios virtuales (VUs) y peticiones por segundo



Figura B. Monitoreo VUs vs. http_reqs provisto en el enunciado (Figura 2 del reto).

El gráfico muestra tres tramos claramente diferenciados:

- **Rampa de subida (≈01:38–01:40):** las VUs suben rápidamente de 0 a ~140, y el throughput las acompaña de forma proporcional, sin señales de estrés.
- **Estado estable con incidente (≈01:40–02:38):** las VUs se mantienen entre ~110 y ~140 con throughput estable, EXCEPTO en una ventana aproximada de 01:50 a 01:56 donde ambas curvas caen de forma abrupta y simultánea (VUs desciende hasta un rango cercano a 0–25, y http_reqs cae en la misma proporción), para luego recuperarse y volver a la línea base ~110–140 VUs. Esta caída sincronizada en ambas métricas es la firma característica de un incidente de disponibilidad del lado del servidor —no de una reducción deliberada de carga generada por el script— y coincide temporalmente con la ventana en la que se concentra el 99.96% de los errores (stage 1).
- **Rampa de bajada (≈02:38–02:41):** ambas curvas descienden a 0 de forma controlada, sin fallos adicionales relevantes.

La caída conjunta de VUs y http_reqs —en vez de una caída de VUs con http_reqs sosteniéndose, o viceversa— es clave: si el generador de carga hubiese reducido intencionalmente la concurrencia, se esperaría una transición más gradual y reproducible en cada corrida. En cambio, el patrón observado (caída abrupta seguida de una recuperación con un pico de rebote) es consistente con un evento de saturación o falla temporal en el sistema bajo prueba, del cual el sistema se recuperó por sí solo (auto-recovery, reinicio de instancia, expiración de un circuito de rate limiting, o normalización de una dependencia).

7. Hallazgos principales

- El SLA de tiempo de respuesta no se cumple en el percentil 95 (1.57s vs. 1.5s permitido), aunque sí se cumple en el percentil 90 (1.28s).
- El 99.96% de los errores se concentra en la etapa de carga sostenida (stage 1), lo que apunta a un problema de capacidad bajo concurrencia sostenida y no a un defecto funcional distribuido uniformemente.
- Existe una ventana temporal (~6 minutos, entre 01:50 y 01:56 aprox.) donde tanto VUs como throughput caen de forma abrupta y simultánea — la evidencia visual más fuerte de un incidente puntual de disponibilidad, coincidente con la concentración de errores de stage 1.
- El valor máximo de latencia (29.93s, ~20x el límite del SLA) indica peticiones que quedaron colgadas, probablemente por ausencia de un timeout de cliente razonable o por saturación del lado del servidor sin fail-fast.
- La tasa de error agregada (2.44%) cumple el SLA (<3%) pero enmascara una probable tasa de error mucho mayor durante la ventana del incidente — el promedio global no es representativo de la experiencia del usuario durante ese lapso.
- Las peticiones fallidas son, en promedio, más lentas que las exitosas (861.68ms vs. 735.84ms), consistente con fallos por timeout más que por rechazo inmediato (fail-fast) del servidor.

8. Conclusiones

El servicio evaluado sostiene un throughput muy por encima del mínimo exigido (73.18 req/s vs. 20 TPS) y, en condiciones normales, responde con tiempos de respuesta razonables (p50~613ms). Sin embargo, la prueba evidencia que bajo carga sostenida (~140 VUs) el sistema experimentó un incidente de disponibilidad concentrado en una ventana acotada de tiempo, que produjo una acumulación de errores HTTP 5xx (y en menor medida 4xx) y latencias extremas (hasta 29.93s). Este incidente es suficiente para incumplir el SLA de tiempo de respuesta en el percentil 95 de la prueba completa, aun cuando el promedio y la tasa de error agregada parezcan aceptables a primera vista.

Desde una perspectiva de negocio, un incidente de esta naturaleza —aunque acotado en el tiempo— puede traducirse en usuarios reales incapaces de iniciar sesión o completar una transacción durante varios minutos, con el consiguiente impacto reputacional y operativo, especialmente si coincide con una ventana de alta demanda real (ej. inicio de jornada laboral, día de pago, campañas). El hallazgo refuerza la importancia de analizar los resultados de performance correlacionando métricas agregadas con series de tiempo y desgloses por etapa, en lugar de basar el veredicto de aceptación únicamente en promedios globales.

9. Recomendaciones

- 1 Investigar la causa raíz del incidente en la ventana ~01:50–01:56: revisar logs de aplicación, métricas de infraestructura (CPU, memoria, GC) y estado de dependencias downstream (base de datos, cache, servicios externos) en ese rango horario específico.
- 2 Definir el SLA de tasa de error y de latencia también sobre ventanas de tiempo acotadas (p. ej. por minuto), además del agregado total, para detectar incidentes puntuales que un promedio de toda la prueba puede diluir y ocultar.

- 3 Establecer timeouts de cliente más conservadores (p. ej. 2–5s) en vez de permitir esperas de hasta 30s, de forma que una petición problemática libere recursos rápido en lugar de agotar el pool de conexiones/hilos y arrastrar a otras peticiones concurrentes.
- 4 Incorporar alertas automáticas sobre el percentil 95 de latencia y sobre tasas de error por ventana deslizando (rolling window), no solo sobre el promedio acumulado, para detectar este tipo de degradación en tiempo real durante pruebas y en producción.
- 5 Repetir la prueba de carga sosteniendo el mismo perfil de VUs (~140) durante un período más prolongado (soak test) y con exportación de métricas por segundo (--out json o integración con Grafana/InfluxDB), para poder cuantificar con precisión la duración e impacto real del incidente en vez de estimarlo visualmente a partir del gráfico de monitoreo.
- 6 Correlacionar la ventana del incidente con el pipeline de despliegues, autoscaling o cualquier evento operativo (deploy, restart, scale-in/out) registrado en ese mismo intervalo de tiempo, ya que la recuperación espontánea observada es compatible con un evento de auto-sanación de infraestructura.

Anexo. Datos de origen

Transcripción íntegra del textSummary.txt provisto en el enunciado (Figura 1):

```
textSummary.txt
1  x App Transaction Balance response was OK
2  L 97% - ✓ 269891 / x 6759
3
4  ■ setup
5
6  ✓ checks.....: 97.55% ✓ 269891 x 6759
7  data_received.....: 842 MB 223 kB/s
8  data_sent.....: 588 MB 156 kB/s
9  failed_request.....: 2.44% ✓ 6759 x 269891
10 http_req_blocked.....: avg=10.97µs min=0s med=0s max=35.02ms p(90)=0s p(95)=0s
11 http_req_connecting.....: avg=3.3µs min=0s med=0s max=11.82ms p(90)=0s p(95)=0s
12 ✓ http_req_duration.....: avg=861.68ms min=191.86ms med=613.42ms max=29.93s p(90)=1.28s p(95)=1.57s
13 ✓ { expected_response:true }.....: avg=735.84ms min=244.92ms med=600.7ms max=26.72s p(90)=1.22s p(95)=1.42s
14 ✓ http_req_failed.....: 2.44% ✓ 6759 x 269891
15 http_req_receiving.....: avg=424.03µs min=0s med=320.7µs max=39.58ms p(90)=988.4µs p(95)=1.05ms
16 http_req_sending.....: avg=43.22µs min=0s med=0s max=31.25ms p(90)=0s p(95)=517µs
17 http_req_tls_handshaking.....: avg=7.36µs min=0s med=0s max=27.02ms p(90)=0s p(95)=0s
18 http_req_waiting.....: avg=861.21ms min=191.86ms med=613.01ms max=29.93s p(90)=1.28s p(95)=1.57s
19 ✓ http_reqs.....: 276650 73.176857/s
20 iteration_duration.....: avg=1.86s min=0s med=1.61s max=30.94s p(90)=2.29s p(95)=2.57s
21 ✓ iterations.....: 276650 73.176857/s
22 vus.....: 2 min=2 max=140
23 vus_max.....: 140 min=140 max=140
24 y_failed_request_stage_0_HTTP5xx...: 1 0.000265/s
25 y_failed_request_stage_1_HTTP4xx...: 769 0.203409/s
26 y_failed_request_stage_1_HTTP5xx...: 5987 1.583625/s
27 y_failed_request_stage_2_HTTP5xx...: 2 0.000529/s
```

Figura 1 (original del enunciado).